Data Mining

Complete Study Notes (Theory and Weka Practical)

Master of Business Administration - Business Analytics Specialisation
Semester III | Amity Business School, Amity University Rajasthan | 2026

Covers Modules I to V of the course profile: introduction and preprocessing, knowledge representation and attribute-oriented analysis, data mining algorithms, cluster analysis, and advanced techniques. Every topic follows the same pattern: definition, explanation, steps or formula, example, and a quick-revision box.

Contents

Course at a Glance 4

Course Information . . . 4
Course Objectives . . . 4
Course Outcomes (CO) . . . 4
Syllabus Map (five modules) . . . 5
Assessment Scheme . . . 5
15-Week Schedule (indicative) . . . 6
Readings . . . 6

Module I: Introduction to Data Mining 7

1.1 What is Data Mining? . . . 7
1.2 Related Technologies . . . 7
1.3 Goals of Data Mining . . . 8
1.4 Stages of the Data Mining Process (KDD) . . . 8
1.5 Data Mining Techniques (overview) . . . 8
1.6 Knowledge Representation Methods . . . 9
1.7 Application Example: the Weather Data . . . 9
1.8 Data Warehouse and DBMS . . . 9
1.9 Multidimensional Data Model . . . 10
1.10 OLAP Operations (Loan Dataset Example) . . . 10
1.11 Data Cleaning . . . 11
1.12 Data Transformation . . . 11
1.13 Data Reduction . . . 12
1.14 Discretisation and Concept Hierarchies . . . 12
1.15 Weka: Installation and Preprocessing Experiments . . . 13

Module II: Knowledge Representation and Attribute-Oriented Analysis 14

2.1 Task-Relevant Data . . . 14
2.2 Background Knowledge . . . 14
2.3 Interestingness Measures . . . 14
2.4 Representing Input Data and Output Knowledge . . . 15
2.5 Visualisation Techniques . . . 15
2.6 Attribute-Oriented Induction (AOI) . . . 15
2.7 Attribute Relevance Analysis . . . 16
2.8 Statistical Measures . . . 17

Module III: Data Mining Algorithms 18

3.1 Association Rule Mining . . . 18
3.2 Classification . . . 19
3.3 Prediction . . . 21

Module IV: Cluster Analysis - Concepts and Methods 23

4.1 Basic Issues in Clustering . . . 23
4.2 The Cluster/2 Conceptual Clustering System . . . 23
4.3 Partitioning Methods . . . 23
4.4 Hierarchical Methods . . . 25
4.5 Density-Based Methods . . . 25
4.6 Grid-Based Methods . . . 26
4.7 Conceptual Clustering: Cobweb . . . 26

Module V: Advanced Techniques, Data Mining Software and Applications 28

5.1 Text Mining . . . 28
5.2 Web Mining . . . 29
5.3 Data Mining Software and Applications . . . 30

Quick Revision Section 31

R1. Formula Sheet . . . 31
R2. Key Comparisons . . . 31
R3. Algorithm Cheat Sheet (steps in one line) . . . 32
R4. Likely Exam Questions . . . 32
R5. Glossary . . . 33

Course at a Glance

Course Information

Item	Detail
Course	Data Mining (Theoretical and Practical)
Programme / Semester	MBA (Business Analytics specialisation), Semester III
School	Amity Business School, Amity University Rajasthan
L-T-P-C	3-0-2-2 (3 lecture hours, 0 tutorial, 2 practical hours, 2 credits); version NEP-V-1.1
Course Coordinator	Yash Yadav (Block F, 2nd Floor, Amity Business School)
Pre-requisites	Basic statistics and spreadsheet operations are helpful but not mandatory
Tool used	Weka 3 (laboratory sessions)
Nature of course	A methods course: preprocessing is taught before any algorithm, because most practical difficulty lies in preparing the data

Course Objectives

- Build a working understanding of data mining foundations and its relationship to machine learning, DBMS and OLAP.
- Develop competence in preprocessing: cleaning, transformation, reduction and discretisation.
- Apply association rule mining, classification, prediction and clustering using an industry-relevant tool (Weka).
- Evaluate mined patterns critically against business relevance and interestingness.
- Extend the techniques to advanced applications such as text mining and web mining.

Course Outcomes (CO)

CO	Statement	Bloom level
CO1	Understand and explain the fundamental concepts, processes and applications of data mining along with related technologies.	L2
CO2	Preprocess and transform raw data to generate suitable input for various data mining techniques and tools.	L2
CO3	Apply and evaluate association rule mining, classification, prediction and clustering using tools like Weka.	L3
CO4	Evaluate complex data types and extract meaningful patterns for business decision-making through advanced data mining applications.	L5

Syllabus Map (five modules)

Module	Title	Main topics	Bloom
I	Introduction to Data Mining	Definition, related technologies, goals, stages, techniques, knowledge representation, weather example, data warehouse and DBMS, multidimensional model, OLAP (loan dataset), cleaning, transformation, reduction, discretisation, concept hierarchies, Weka installation and filters	L2
II	Knowledge Representation and Attribute-Oriented Analysis	Task-relevant data, background knowledge, interestingness, representing input and output, visualisation, attribute generalisation, attribute relevance, class comparison, statistical measures	L3
III	Data Mining Algorithms	Association rules (item sets, Apriori, correlation), classification (1R, decision trees, covering rules), prediction (Bayesian, Bayesian networks, nearest neighbour, linear models)	L3
IV	Cluster Analysis	Basic issues, Cluster/2, k-means, EM, hierarchical (agglomerative, divisive), density-based, grid-based, Cobweb	L3
V	Advanced Techniques, Software and Applications	Text mining (keywords, parsing, Bayesian text classification), web mining, data mining software and applications	L4, L5

Assessment Scheme

Total 100 marks: Continuous Assessment (formative) 50 and End Semester Assessment (summative) 50.

Component	Marks / weightage	Indicative timing
Internal Practical (IP): Weka lab record	5	Continuous, weeks 3 to 9
Class Tests (CT)	25	Week 8
Project (P): capstone mining exercise	10	Weeks 9 to 10
External Practical (EP): viva on project and lab work	5	Week 10
Attendance (A)	5	Last week of teaching (minimum ordinarily 75%)
End Semester Examination (EE)	50	As per university schedule

Practical and viva marking rewards preprocessing rigour (justify every cleaning or discretisation step), tool proficiency, and the ability to explain rules, trees and clusters in plain business language. Class tests reward conceptual accuracy and choosing the right technique for a given dataset.

15-Week Schedule (indicative)

Week	Topic	CO
1	What is data mining; related technologies; goals and stages	CO1
2	Knowledge representation; data warehouse and DBMS; OLAP operations	CO1
3	Data cleaning, transformation, reduction; installing Weka 3; filters	CO1
4	Task-relevant data, background knowledge, interestingness, visualisation (Weka lab)	CO2
5	Attribute-oriented analysis: generalisation, relevance, class comparison	CO2
6	Statistical measures and class comparison; class test preparation	CO2
7	Association rule mining (Weka lab)	CO3
8	Classification: 1R, decision trees, covering rules (Weka lab)	CO3
9	Prediction: Bayesian, Bayesian networks, instance-based (Weka lab)	CO3
10	Cluster analysis: partitioning, k-means, EM (Weka lab)	CO3
11	Hierarchical, density-based, grid-based clustering; Cobweb (Weka lab)	CO3
12	Text mining, keyword extraction, Bayesian classification	CO4
13	Web mining; knowledge extraction from the web; data mining software	CO4
14	Project development; lab record consolidation	CO3, CO4
15	Project presentation; viva-voce; external practical	CO4

Readings

- Textbook: Han, J., Kamber, M. and Pei, J. (2011). Data Mining: Concepts and Techniques, 3rd ed., Morgan Kaufmann.
- References: Berson and Smith (Data Warehousing, Data Mining and OLAP); Marakas (Modern Data Warehousing, Mining and Visualization); Berry and Linoff (Data Mining Techniques); Sharma (Data Mining, Data Warehousing and OLAP); Gupta (Data Mining with Case Studies); Ponniah (Data Warehousing Fundamentals).

Module I: Introduction to Data Mining

1.1 What is Data Mining?

Definition

Data mining is the process of discovering interesting, previously unknown and potentially useful patterns, relationships and knowledge from large volumes of data using statistical, machine learning and database techniques.

Explanation

- The name is a slight misnomer: we mine *knowledge* from data, just as gold is mined from rock. A more accurate term is **Knowledge Discovery from Data (KDD)**.
- Data mining is one core step of KDD, where intelligent methods extract patterns. Many authors use the two terms interchangeably.
- Why it matters: organisations collect huge data (sales, web logs, sensors, transactions) but are "data rich and information poor". Data mining converts data into decisions.
- Patterns must be valid on new data, novel, useful, and understandable to be called knowledge.

Business examples

- Retail: which products are bought together (market basket analysis).
- Banking: classify loan applicants as low or high risk; detect fraudulent transactions.
- Telecom: cluster customers and predict churn.
- Marketing: segment customers and target campaigns.

QUICK REVISION
• Data mining = discovering valid, novel, useful, understandable patterns from large data.
• KDD is the full process; data mining is its pattern-discovery step.
• Descriptive mining finds patterns (clusters, associations); predictive mining forecasts (classification, prediction).

1.2 Related Technologies

Data mining sits at the intersection of several fields. Exam answers should show how it relates to and differs from each.

Technology	Main purpose	Relation to data mining
Machine Learning (ML)	Builds algorithms that learn from data and improve with experience.	Supplies many mining algorithms (decision trees, clustering, Bayes). ML focuses on learning accuracy; data mining focuses on discovering usable knowledge from large, real databases.
DBMS	Stores, retrieves and manages data efficiently; answers known queries (SQL).	Provides the data source. A query retrieves what you ask for; mining discovers patterns you did not know to ask for.
OLAP	Interactive multidimensional analysis (roll-up, drill-down, slice, dice).	OLAP is user-driven and verifies hypotheses; data mining is discovery-driven and finds hidden patterns automatically.
Statistics	Collects, analyses and interprets data; hypothesis testing and estimation.	Provides the mathematical base. Statistics usually works on small samples and tests a hypothesis; mining works on large data and generates hypotheses.
Data warehouse	Integrated, historical, subject-oriented store for analysis.	The ideal clean source of data for mining.

QUICK REVISION
• ML = algorithms that learn; DBMS = store and query; OLAP = interactive multidimensional views; Statistics = inference and testing.
• OLAP and queries are verification-driven; data mining is discovery-driven.

1.3 Goals of Data Mining

- **Description:** summarise and characterise the data (what is going on?).
- **Prediction:** forecast unknown or future values from known data (what will happen?).
- **Pattern discovery:** find associations, clusters, sequences and outliers.
- **Decision support:** turn patterns into business actions such as targeted offers, risk control and cost reduction.
- **Verification and exploration:** test hypotheses and explore unfamiliar data.

Two broad families: **descriptive** (characterisation, association, clustering) and **predictive** (classification, regression/prediction).

1.4 Stages of the Data Mining Process (KDD)

1. **Data cleaning:** remove noise and inconsistent data, handle missing values.
2. **Data integration:** combine multiple sources (databases, files) into one consistent store.
3. **Data selection:** retrieve the data relevant to the analysis task.
4. **Data transformation:** normalise, aggregate or consolidate into forms suitable for mining.
5. **Data mining:** apply intelligent methods to extract patterns.
6. **Pattern evaluation:** identify the truly interesting patterns using interestingness measures.
7. **Knowledge presentation:** visualise and present the results to users.

Steps 1 to 4 together are **data preprocessing**. In practice the process is iterative: results often send the analyst back to earlier steps.

QUICK REVISION
• Mnemonic: Clean, Integrate, Select, Transform, Mine, Evaluate, Present.
• Preprocessing (steps 1 to 4) usually consumes most of the project time.

1.5 Data Mining Techniques (overview)

Technique	Task	Example
Association rule mining	Find items or events that occur together	Customers who buy bread also buy milk
Classification	Assign records to predefined classes (supervised)	Loan applicant is low or high risk
Prediction / regression	Estimate a numeric value	Forecast next month sales
Clustering	Group similar records without predefined labels (unsupervised)	Customer segmentation
Outlier / anomaly detection	Find records that deviate from the norm	Credit card fraud
Characterisation and comparison	Summarise a class or compare classes	Profile of high-value customers vs others
Text and web mining	Mine unstructured text and web data	Sentiment of reviews, page classification

1.6 Knowledge Representation Methods

Mined knowledge must be expressed in a form people can read and systems can use.

Method	Description
Tables	Simplest form; a lookup table of attribute values and outcomes (also used by 1R).
Decision trees	Tree of attribute tests leading to class labels at the leaves; easy to read.
Classification (if-then) rules	IF conditions THEN class; may be ordered as a decision list.
Association rules	A => B with support and confidence.
Linear models	Numeric equation, for example y = w0 + w1x1 + w2x2.
Instance-based representation	Store examples and classify by similarity (nearest neighbour).
Clusters	Groups of instances: partitions, hierarchies (dendrograms), or overlapping/probabilistic clusters.

1.7 Application Example: the Weather Data

The weather dataset (14 days) is the classic teaching dataset. Each row is a day described by four attributes, and the class attribute **play** says whether golf/tennis was played. It is used throughout this course for rules, trees, Bayes and Weka labs.

#	Outlook	Temperature	Humidity	Windy	Play
1	sunny	hot	high	false	no
2	sunny	hot	high	true	no
3	overcast	hot	high	false	yes
4	rainy	mild	high	false	yes
5	rainy	cool	normal	false	yes
6	rainy	cool	normal	true	no
7	overcast	cool	normal	true	yes
8	sunny	mild	high	false	no
9	sunny	cool	normal	false	yes
10	rainy	mild	normal	false	yes
11	sunny	mild	normal	true	yes
12	overcast	mild	high	true	yes
13	overcast	hot	normal	false	yes
14	rainy	mild	high	true	no

Class distribution: 9 yes and 5 no. A rule learnt from it: *IF outlook = overcast THEN play = yes* (true for all 4 overcast days). Business parallel: replace the weather attributes with customer attributes and "play" with "buys / does not buy".

1.8 Data Warehouse and DBMS

Definition

A **data warehouse** is a subject-oriented, integrated, time-variant and non-volatile collection of data that supports management decision making (W. H. Inmon).

- **Subject-oriented:** organised around subjects such as customer, product, sales rather than applications.
- **Integrated:** data from many sources is made consistent (naming, units, encoding).
- **Time-variant:** holds historical data (for example 5 to 10 years) with a time dimension.

- **Non-volatile:** data is loaded and read, not updated or deleted in day-to-day operation.

Operational DBMS (OLTP) vs Data Warehouse (OLAP)

Feature	Operational DBMS (OLTP)	Data warehouse (OLAP)
Purpose	Day-to-day transactions	Analysis and decision support
Users	Clerks, DBAs, operational staff	Managers, analysts, executives
Data	Current, detailed, normalised	Historical, summarised, consolidated
Design	Entity-relationship, application-oriented	Star or snowflake schema, subject-oriented
Access	Short read/write transactions	Mostly complex read-only queries
Size	Megabytes to gigabytes	Gigabytes to terabytes and more

QUICK REVISION
• Warehouse = subject-oriented + integrated + time-variant + non-volatile.
• OLTP runs the business; OLAP analyses the business.

1.9 Multidimensional Data Model

Warehouses model data as a **data cube**: a measure (numeric fact such as sales amount) viewed along several **dimensions** (time, product, location, customer). Each dimension may have a **concept hierarchy** such as day < month < quarter < year.

- **Fact table:** central table with measures and foreign keys to dimensions.
- **Dimension tables:** descriptive attributes of each dimension.
- **Star schema:** one fact table linked directly to denormalised dimension tables (simple, fast).
- **Snowflake schema:** dimension tables are normalised into sub-tables (saves space, more joins).
- **Fact constellation (galaxy):** several fact tables share dimension tables.
- **Cuboid:** each combination of dimensions is a cuboid; all cuboids form a lattice. The apex cuboid is the grand total; the base cuboid is the lowest level.

Measure types: **distributive** (sum, count, min, max), **algebraic** (average), **holistic** (median, rank).

1.10 OLAP Operations (Loan Dataset Example)

Take a bank loan cube with dimensions **Time** (month, quarter, year), **Branch / Location** (city, state), **Loan type** (home, car, personal) and measure **loan amount**.

Operation	Meaning	Loan example
Roll-up (drill-up)	Aggregate: climb a hierarchy or remove a dimension	Loan amount by city rolled up to loan amount by state
Drill-down	Opposite of roll-up: go to more detail	Quarter totals drilled down to monthly totals
Slice	Select one value on one dimension, giving a sub-cube	Only loan type = home
Dice	Select values on two or more dimensions	Loan type in (home, car) and year in (2025, 2026) and city in (Jaipur, Delhi)
Pivot (rotate)	Rotate axes to view data from another angle	Swap rows (branch) and columns (loan type)

QUICK REVISION
• Roll-up = less detail; drill-down = more detail; slice = fix one dimension; dice = fix several; pivot = rotate.
• OLAP operations support interactive exploration; data mining discovers patterns automatically.

1.11 Data Cleaning

Definition

Data cleaning (data cleansing) is the process of detecting and correcting or removing errors, noise, inconsistencies and missing values so that data is accurate, complete and consistent. Real-world data is dirty: incomplete, noisy and inconsistent. "Garbage in, garbage out": poor quality data gives misleading patterns.

Why it is important

- Mining algorithms assume reasonably clean input; errors produce wrong or meaningless patterns.
- Quality decisions need quality data; cleaning typically takes the largest share of project effort.
- Clean, consistent data improves model accuracy and makes results trustworthy for business decisions.

Main problems and methods

Problem	Methods
Missing values	1) Ignore the tuple (poor if many missing). 2) Fill manually (slow). 3) Use a global constant such as "Unknown". 4) Use the attribute mean or median (class-wise mean is better). 5) Use the most probable value (regression, Bayes, decision tree).
Noisy data (random error)	**Binning** (smooth by bin means, medians or boundaries); **regression** (fit a function); **clustering** (outliers fall outside clusters); combined computer and human inspection.
Inconsistent data	Correct using rules, constraints, external references (for example Age = 5 but Birth year 1990); standardise formats and codes.
Duplicates / redundancy	Remove duplicate records; resolve entity identification.

EXAMPLE: Smoothing by binning

• Sorted prices: 4, 8, 15, 21, 21, 24, 25, 28, 34 (equal-frequency bins of 3).
• Bin 1: 4, 8, 15; Bin 2: 21, 21, 24; Bin 3: 25, 28, 34.
• Smoothing by bin means: Bin 1 = 9, 9, 9; Bin 2 = 22, 22, 22; Bin 3 = 29, 29, 29.
• Smoothing by bin boundaries (each value moves to the nearer boundary): Bin 1 = 4, 4, 15; Bin 2 = 21, 21, 24; Bin 3 = 25, 25, 34.

QUICK REVISION

• Cleaning handles missing values, noise and inconsistency.
• Binning, regression and clustering smooth noisy data.
• Important because dirty data gives unreliable models; it precedes all other preprocessing.

1.12 Data Transformation

Converts data into forms suitable for mining.

- **Smoothing:** remove noise (binning, regression, clustering).
- **Aggregation:** summarise, for example daily sales to monthly sales.
- **Generalisation:** replace low-level data by higher-level concepts using hierarchies (city to country).
- **Normalisation:** scale values into a small range so large-valued attributes do not dominate (needed for distance-based methods such as k-means and k-NN).
- **Attribute / feature construction:** build new attributes from existing ones (for example area = length x width).

Normalisation formulas

```
Min-max: v' = (v - min) / (max - min) x (new_max - new_min) + new_min

Z-score: v' = (v - mean) / standard_deviation
```

```
Decimal scaling: v' = v / 10^j (j = smallest integer so that max |v'| < 1)
```

EXAMPLE: Normalising income
- Income range 12,000 to 98,000; value 73,600.
- Min-max to [0,1]: (73,600 - 12,000) / (98,000 - 12,000) = 61,600 / 86,000 = 0.716.
- Z-score with mean 54,000 and standard deviation 16,000: (73,600 - 54,000) / 16,000 = 1.225.
- Decimal scaling: if the maximum absolute value is 986, j = 3, so -986 becomes -0.986.

1.13 Data Reduction

Data reduction obtains a smaller representation of the data that gives the same or almost the same analytical results, making mining faster on huge data.

Strategy	Idea
Data cube aggregation	Aggregate to the level needed (daily to quarterly).
Attribute subset selection	Remove irrelevant or redundant attributes (forward selection, backward elimination, decision-tree induction, information gain).
Dimensionality reduction	Transform to fewer dimensions: Principal Component Analysis (PCA), wavelet transforms.
Numerosity reduction	Replace data by smaller models: regression, histograms, clustering, sampling.
Sampling	Mine a representative subset (simple random, stratified, cluster sampling).
Data compression	Lossless or lossy encoding of data.

1.14 Discretisation and Concept Hierarchies

Definition

Discretisation divides the range of a continuous attribute into intervals and replaces values with interval labels (for example age 23 becomes "20-30" or "young"). A **concept hierarchy** organises values at multiple levels of abstraction, for example street < city < state < country, or age: 0-20 = youth, 21-60 = adult, above 60 = senior.

Discretisation methods

- **Equal-width binning:** intervals of the same size (simple, sensitive to outliers).
- **Equal-frequency (equal-depth) binning:** each interval holds about the same number of values.
- **Histogram analysis:** uses histogram buckets; unsupervised.
- **Clustering-based:** cluster values and treat clusters as intervals.
- **Entropy-based (supervised):** choose split points that maximise information gain with respect to the class.
- **Natural partitioning (3-4-5 rule):** creates intuitive intervals.

Why discretise: some algorithms (such as 1R, Apriori, Naive Bayes in nominal form) need categorical attributes; it also reduces data size and makes patterns easier to interpret. Concept hierarchies enable mining at multiple levels, roll-up and drill-down, and attribute-oriented induction.

QUICK REVISION
- Discretisation turns numeric into categorical; concept hierarchies climb from detailed to general values.
- Equal-width vs equal-frequency; supervised (entropy-based) vs unsupervised.

1.15 Weka: Installation and Preprocessing Experiments

Weka (Waikato Environment for Knowledge Analysis) is a free, open-source Java toolkit from the University of Waikato (New Zealand) with a collection of data preprocessing and machine learning algorithms and a graphical interface.

Installing Weka 3

1. Download Weka 3 (stable version) for your operating system from the University of Waikato Weka website.
2. Run the installer (Windows installers can include Java; otherwise install a suitable Java runtime first).
3. Launch Weka and click **Explorer** in the GUI Chooser.
4. Click Open file in the Preprocess tab and load a dataset (for example weather.nominal.arff from the Weka data folder).

Explorer tabs

Tab	Use
Preprocess	Load data, view attributes, apply filters, edit data
Classify	Train and test classifiers (ZeroR, OneR, J48, NaiveBayes, IBk, LinearRegression)
Cluster	SimpleKMeans, EM, Cobweb, hierarchical clusterers
Associate	Apriori and other association rule learners
Select attributes	Attribute evaluators and search methods (for example InfoGain with Ranker)
Visualize	Scatter plot matrix and plots of attributes

Data format and filters

- Native format is **ARFF** (Attribute-Relation File Format): a header with @relation and @attribute lines, then @data rows. Weka also reads CSV.
- Filters are in the Preprocess tab, split into **supervised** (use the class) and **unsupervised**, and into attribute and instance filters.
- Useful filters: **ReplaceMissingValues** (fills with mean/mode), **Discretize** (equal-width or equal-frequency bins), **Normalize** (scale to 0-1), **Standardize** (z-score), **Remove** (drop attributes), **NominalToBinary**, **Resample**.

EXAMPLE: Lab: discretising an attribute

• Load a dataset with a numeric attribute such as temperature, choose filters > unsupervised > attribute > Discretize, set bins (for example 3) and Apply.
• The numeric attribute becomes nominal intervals such as (-inf-69.5], (69.5-77.5], (77.5-inf).
• Record in the lab record what you did and why: preprocessing justification is graded.

Module II: Knowledge Representation and Attribute-Oriented Analysis

2.1 Task-Relevant Data

Users rarely mine the whole database. **Task-relevant data** is the portion of the database that matters for the mining task, specified through a data mining query (a primitive).

- **Database or warehouse name** and the tables or cubes involved.
- **Conditions for selecting the data** (for example region = Rajasthan and year = 2026).
- **Relevant attributes or dimensions** to include.
- **Grouping criteria** where needed.

Specifying the task precisely reduces the amount of data mined, speeds up mining, and avoids irrelevant patterns. The kind of knowledge to be mined (characterisation, comparison, association, classification, clustering) is also part of the task definition.

2.2 Background Knowledge

Background knowledge is information about the domain that guides the mining process and the evaluation of patterns. The most common form is the **concept hierarchy**, which lets data be mined at several levels of abstraction.

- **Schema hierarchy:** total or partial order among attributes, for example street < city < state < country.
- **Set-grouping hierarchy:** groups of values, for example {20..39} = young, {40..59} = middle-aged.
- **Operation-derived hierarchy:** built by data analysis, clustering or parsing.
- **Rule-based hierarchy:** defined by rules, for example a low-profit item is one with margin below 50 rupees.
- Other background knowledge: domain constraints and user beliefs used to judge whether a pattern is novel.

2.3 Interestingness Measures

A mining algorithm can produce thousands of patterns, and most are uninteresting. A pattern is interesting if it is **(1) easily understood, (2) valid on new data with some certainty, (3) potentially useful, and (4) novel** (or validates a hypothesis).

Type	Measures	Meaning
Objective (data-driven)	Support, confidence, lift, accuracy, coverage	Based on the statistics of the pattern itself
Subjective (user-driven)	Unexpectedness, novelty, actionability	Depends on the user and the business goal
Simplicity	Rule length, tree size	Shorter patterns are easier to understand
Certainty	Confidence, classification reliability	How likely the pattern is to hold
Utility	Support, profit impact	How useful or valuable the pattern is

Interestingness can be used both after mining (filter patterns) and during mining (prune the search). The course objective stresses evaluating output **against business relevance** instead of treating it as automatically useful.

QUICK REVISION
• Interesting pattern = understandable, valid, useful, novel.
• Objective measures: support, confidence, lift. Subjective: novelty, actionability, unexpectedness.

2.4 Representing Input Data and Output Knowledge

Input: concepts, instances and attributes

- **Instance (example, record, tuple):** one individual thing to be classified, associated or clustered (a row).
- **Attribute (feature, field):** a measured characteristic of an instance (a column). One may be designated the **class** attribute.
- **Concept to be learned:** the thing to be predicted or described.

Attribute type	Description	Example	Allowed operations
Nominal (categorical)	Names or labels, no order	outlook = sunny, rainy	Equality only
Ordinal	Ordered categories, differences not meaningful	cool < mild < hot; small, medium, large	Order comparison
Interval	Numeric, equal spacing, no true zero	Temperature in Celsius, year	Add, subtract
Ratio	Numeric with true zero	Income, weight, age	All arithmetic, ratios

Input preparation issues: gathering data from many sources, ARFF format, missing values (a value that is unknown or unrecorded), inaccurate values and identifying data errors.

Output: knowledge representation

Output can be tables, linear models, decision trees, classification rules, association rules, rules with exceptions, instance-based representations (prototypes) and clusters. Choosing the right output representation affects how well business users can act on it.

2.5 Visualisation Techniques

Data visualisation uses graphics to reveal patterns, outliers and relationships quickly. It is used to understand data before mining and to present results after mining.

Technique	Use
Histogram	Distribution of one numeric attribute
Bar and pie chart	Compare categories or proportions
Scatter plot / scatter-plot matrix	Relationship between two attributes; correlation, clusters, outliers
Box plot	Five-number summary; spots outliers
Line chart	Trends over time
Parallel coordinates / dashboards	Many attributes at once; business reporting
Tree, cluster and dendrogram views	Showing mined models

EXAMPLE: Weka visualisation experiment
• Preprocess tab: select an attribute and view its histogram, coloured by the class.
• Visualize tab: scatter-plot matrix of all attributes; click a cell to enlarge and look for patterns or outliers.
• Classify tab: right-click a result and choose Visualize tree or Visualize classifier errors.

2.6 Attribute-Oriented Induction (AOI)

Definition

Attribute-oriented induction is a set-oriented, data-generalisation method of **concept description**. It collects the task-relevant data, then **generalises** it by replacing low-level attribute values with higher-level concepts (using concept hierarchies) and merging identical tuples, accumulating a **count**. The result is a compact **generalised relation** that characterises the data in simple language.

Explain in simple language

Imagine a table of students with exact birthplaces and exact marks. AOI "zooms out": cities become countries, marks become Low/Medium/High, and identical rows are merged with a count. You then see the big picture, for example "most students are from India with High grades".

Steps of the AOI algorithm

1. **Data collection:** fetch the task-relevant data (initial relation) with a query.
2. **Attribute removal:** drop an attribute if it has many distinct values but no concept hierarchy, or if its higher-level concepts are expressed by other attributes.
3. **Attribute generalisation:** if an attribute has many distinct values and a hierarchy exists, replace values with their higher-level concepts (climb one level at a time).
4. **Generalisation control:** use an **attribute generalisation threshold** (maximum distinct values per attribute) and a **relation threshold** (maximum tuples in the final relation).
5. **Aggregation:** merge identical generalised tuples and add their counts (a **count** or **vote** attribute).
6. **Presentation:** show as generalised relation, cross-tabulation, charts or generalised rules (for example *major(x) = Science and birthplace(x) = Canada implies grade(x) = Excellent [60%]*).

EXAMPLE: Student data

• Initial relation: Name, Gender, Major, Birth_place, GPA.
• Name: many distinct values and no hierarchy, so it is removed. Major: Physics, Chemistry, Biology go to Science; CS, Electrical go to Engineering.
• Birth_place: Vancouver, Toronto go to Canada; Jaipur, Delhi go to India. GPA: 3.7 to 4.0 = Excellent; 3.3 to 3.6 = Very good.
• After merging: (Science, Canada, Excellent) count 16; (Engineering, India, Very good) count 22; and so on.

Class characterisation and class comparison

- **Characterisation:** summarise the general features of a target class (for example profile of graduate students).
- **Class comparison (discrimination):** compare the target class with one or more contrasting classes. Two measures: **t-weight** (share of a tuple within its own class) and **d-weight** (share of the tuple in the target class compared with all classes), where d-weight = count in target class / total count across target and contrasting classes.

QUICK REVISION

• AOI = collect, remove, generalise with concept hierarchies, merge with counts, present.
• Two thresholds control generalisation: attribute threshold and relation threshold.
• Output: generalised relation, cross-tab or generalised rule. Used for characterisation and comparison.

2.7 Attribute Relevance Analysis

Before generalising or building models, remove attributes that carry little information about the class. **Attribute relevance analysis** ranks attributes by how well they separate classes.

- **Information gain** (entropy reduction), gain ratio, **Gini index**, **chi-square test**, correlation coefficient.
- Steps: data collection, preliminary relevance analysis (using a measure and a threshold), then mining with the relevant attributes only.
- Benefits: fewer dimensions, faster and often more accurate mining, easier interpretation.
- In Weka: the Select attributes tab with InfoGainAttributeEval and the Ranker search method.

2.8 Statistical Measures

Descriptive statistics give the first understanding of the data and detect outliers and skew.

Type	Measures
Central tendency	Mean, weighted mean, median, mode, midrange
Dispersion	Range, quartiles (Q1, Q3), inter-quartile range IQR = Q3 - Q1, variance, standard deviation
Five-number summary	Minimum, Q1, median, Q3, maximum (shown in a box plot)
Shape	Skewness (positive, negative), symmetric distribution
Relationship	Covariance, correlation coefficient

```
Mean = (sum of x) / n Variance = sum (x - mean)^2 / n Std dev = square root of variance

Correlation r = sum (x - xbar)(y - ybar) / ( n x sx x sy ) r in [-1, +1]
```

Rule of thumb for outliers: a value is suspect if it lies more than 1.5 x IQR below Q1 or above Q3. If mean > median the distribution is positively (right) skewed. In Weka, the attribute panel in the Preprocess tab shows minimum, maximum, mean and standard deviation.

Module III: Data Mining Algorithms

3.1 Association Rule Mining

Motivation

Association rule mining discovers interesting relationships among items in large transaction databases. The classic motivation is **market basket analysis**: which products are bought together? It guides shelf placement, cross-selling, promotions and catalogue design.

Terminology

Term	Meaning
Item / itemset	An item is a product; an itemset is a set of items. A k-itemset has k items.
Transaction	One basket (set of items bought together).
Association rule	Implication A => B where A and B are disjoint itemsets (A = antecedent, B = consequent).
Support count	Number of transactions that contain the itemset.
Support	Fraction of all transactions that contain A and B together: how frequent the rule is.
Confidence	Of the transactions containing A, the fraction that also contain B: how reliable the rule is.
Frequent itemset	Itemset whose support is at least the minimum support threshold.
Strong rule	Rule that satisfies both minimum support and minimum confidence.
Lift	How much more often A and B occur together than expected if independent.

```
Support(A => B) = P(A and B) = count(A and B) / N

Confidence(A => B) = P(B | A) = support(A and B) / support(A)

Lift(A => B) = confidence(A => B) / support(B) = P(A and B) / ( P(A) x P(B) )
```

Generating item sets and rules efficiently: the Apriori algorithm

Apriori property: every non-empty subset of a frequent itemset must also be frequent. Equivalently, if an itemset is infrequent, all its supersets are infrequent, so they can be pruned without counting. This makes the search efficient.

1. Set minimum support and minimum confidence.
2. Scan the database, count each item, and keep those meeting minimum support: **L1** (frequent 1-itemsets).
3. **Join step:** join Lk-1 with itself to generate candidate k-itemsets **Ck**.
4. **Prune step:** remove any candidate that has an infrequent (k-1)-subset (Apriori property).
5. Scan the database to count the remaining candidates and keep those meeting minimum support: **Lk**.
6. Repeat steps 3 to 5 until no new frequent itemsets are found.
7. **Rule generation:** for every frequent itemset l and every non-empty proper subset s, output the rule s => (l - s) if its confidence meets the minimum confidence.

EXAMPLE: Apriori on 5 transactions (minimum support count = 3, i.e. 60%)
• T1: Bread, Milk | T2: Bread, Diaper, Beer, Eggs | T3: Milk, Diaper, Beer, Coke | T4: Bread, Milk, Diaper, Beer | T5: Bread, Milk, Diaper, Coke.
• C1 counts: Bread 4, Milk 4, Diaper 4, Beer 3, Coke 2, Eggs 1. L1 = {Bread, Milk, Diaper, Beer} (Coke and Eggs dropped).
• C2 counts: Bread-Milk 3, Bread-Diaper 3, Bread-Beer 2, Milk-Diaper 3, Milk-Beer 2, Diaper-Beer 3. L2 = {Bread, Milk}, {Bread, Diaper}, {Milk, Diaper}, {Diaper, Beer}.
• C3: {Bread, Milk, Diaper} has all subsets frequent but its count is only 2 (T4, T5), so it fails. Other candidates are pruned because they contain {Bread, Beer} or {Milk, Beer}. L3 is empty, so stop.
• Rules from {Diaper, Beer}: Beer => Diaper has support 3/5 = 60% and confidence 3/3 = 100%; Diaper => Beer has confidence 3/4 = 75%.
• Lift (Beer => Diaper) = 1.00 / 0.80 = 1.25, greater than 1, so a positive association.

Improvements to Apriori

- Hash-based itemset counting, transaction reduction, partitioning and sampling reduce database scans.
- **FP-Growth** avoids candidate generation by compressing the database into a frequent-pattern tree (FP-tree) and mining it recursively.
- Limitations of Apriori: many database scans and a huge number of candidates for low support thresholds.

Correlation analysis

High support and confidence can still be misleading (for example a rule about an item that almost everyone buys). Correlation analysis checks whether A and B are really related.

- **Lift > 1:** positive correlation; **lift = 1:** independent; **lift < 1:** negative correlation.
- **Chi-square test** also measures dependence between A and B.
- So rules are judged by support, confidence *and* a correlation measure.

Weka: load a nominal dataset (such as weather.nominal.arff or supermarket.arff), go to the Associate tab, choose Apriori, set lowerBoundMinSupport, minMetric (confidence) and numRules, and click Start. Weka lists the best rules with confidence.

QUICK REVISION
• Support = frequency; confidence = reliability; lift = strength beyond chance.
• Apriori: L1, join, prune (Apriori property), count, repeat; then generate rules from frequent itemsets.
• Apriori needs many scans; FP-Growth avoids candidate generation.

3.2 Classification

Basic learning/mining tasks

Classification is **supervised learning**: build a model from training data whose class labels are known and use it to predict the class of new, unseen records. Two steps: (1) **learning** (model construction on training data) and (2) **classification** (testing the model on separate test data, then using it). Typical uses: credit risk, churn, fraud, medical diagnosis.

- Training set builds the model; test set measures accuracy. Avoid overfitting (model too specific to the training data).
- Evaluation methods: hold-out, k-fold cross-validation (Weka default 10-fold), leave-one-out.

Evaluation measures (confusion matrix)

	Predicted positive	Predicted negative
Actual positive	True Positive (TP)	False Negative (FN)
Actual negative	False Positive (FP)	True Negative (TN)

```
Accuracy = (TP + TN) / (TP + TN + FP + FN) Precision = TP / (TP + FP)

Recall (sensitivity) = TP / (TP + FN) F-measure = 2 x Precision x Recall /
(Precision + Recall)
```

1R algorithm (One Rule)

1R builds a one-level decision rule: for each attribute make a rule that predicts the majority class for each attribute value, calculate the error rate of each attribute's rule set, and choose the attribute with the smallest total error.

1. For each attribute, for each of its values: count the class frequencies and predict the most frequent class.
2. Count the errors (instances not in the majority class) for that attribute's rules.
3. Choose the attribute with the lowest total error; break ties arbitrarily (or choose the first).
4. Numeric attributes are first discretised (each interval must hold a minimum number of majority-class instances).

EXAMPLE: 1R on the weather data (14 instances)
• Outlook: sunny => no (2 errors of 5), overcast => yes (0 of 4), rainy => yes (2 of 5); total 4/14.
• Temperature: hot => no or yes tie (2/4), mild => yes (2/6), cool => yes (1/4); total 5/14.
• Humidity: high => no (3/7), normal => yes (1/7); total 4/14. Windy: false => yes (2/8), true => tie (3/6); total 5/14.
• Outlook and humidity tie at 4/14; choosing outlook gives the rule set above. Accuracy on training data is 10/14, about 71%.

Why 1R matters: it is very simple, fast and often surprisingly accurate, so it is a useful baseline. In Weka it is **OneR** (and **ZeroR** predicts the majority class only).

Decision trees

A **decision tree** is a flowchart-like tree: each internal node tests an attribute, each branch is an outcome of the test, and each leaf holds a class label. It is built by **divide and conquer**: choose the best splitting attribute, split the data, and recurse on each subset until the subsets are pure or no attributes remain.

- **ID3** uses information gain; **C4.5** (Weka **J48**) uses gain ratio and handles numeric attributes, missing values and pruning; **CART** uses the Gini index.
- **Pruning** (pre-pruning and post-pruning) removes unreliable branches to prevent overfitting.
- Advantages: easy to interpret and convert into IF-THEN rules; handles mixed data.

```
Entropy: Info(D) = - sum over classes pi x log2(pi)

Info_A(D) = sum over values j of ( |Dj| / |D| ) x Info(Dj)

Information Gain: Gain(A) = Info(D) - Info_A(D)

Gain Ratio = Gain(A) / SplitInfo(A) Gini(D) = 1 - sum pi^2
```

EXAMPLE: Choosing the root of the weather tree
• Info(D) with 9 yes and 5 no = -(9/14)log2(9/14) - (5/14)log2(5/14) = 0.940 bits.
• Outlook: sunny (2 yes, 3 no) = 0.971; overcast (4 yes, 0 no) = 0; rainy (3 yes, 2 no) = 0.971. Weighted = (5/14)(0.971) + (4/14)(0) + (5/14)(0.971) = 0.694. Gain(outlook) = 0.940 - 0.694 = about 0.247.
• Gains: outlook 0.247, humidity 0.152, windy 0.048, temperature 0.029. Outlook is the best root.
• Overcast is pure (always yes). Sunny splits further on humidity (high = no, normal = yes); rainy splits on windy (false = yes, true = no).

Covering rules (separate-and-conquer)

Covering algorithms (for example **PRISM**) learn rules one class at a time: build a rule that covers as many positive examples of the class as possible, remove the instances it covers, and repeat. Unlike a decision tree (divide and conquer, looks at all classes at once), a covering algorithm focuses on one class at a time.

1. For each class, start with all instances.
2. Create a rule with an empty condition: IF ? THEN class = c.
3. Add the attribute-value test that maximises the accuracy p/t (p = positive covered, t = total covered); ties go to higher coverage p.
4. Keep adding tests until the rule is perfect (accuracy 100%) or no attributes remain.
5. Remove the covered instances and repeat until all instances of the class are covered; then move to the next class.

EXAMPLE: First PRISM rule for play = yes
• Candidate tests: outlook = overcast has accuracy 4/4, the best. Rule: IF outlook = overcast THEN play = yes.
• Remove the 4 overcast instances and continue for the remaining yes instances (for example humidity = normal and windy = false).

Aspect	Decision tree	Covering rules (PRISM)
Strategy	Divide and conquer	Separate and conquer
Focus	All classes at once	One class at a time
Output	Tree (convertible to rules)	Ordered or unordered if-then rules
Weka	J48	PRISM, JRip, PART

QUICK REVISION
• Classification = supervised; build model on training data, test on unseen data.
• 1R: one attribute, pick lowest-error attribute. Decision tree: split by highest information gain. Covering: build rules class by class.
• Accuracy, precision, recall and F-measure come from the confusion matrix.

3.3 Prediction

In this syllabus "prediction" covers methods that predict class labels or numeric values: statistical (Bayesian) classification, Bayesian networks, instance-based learning and linear models.

Statistical (Bayesian) classification: Naive Bayes

Bayesian classifiers predict the probability that a record belongs to a class using **Bayes theorem**. **Naive Bayes** assumes attributes are **conditionally independent** given the class (the "naive" assumption), which makes it simple and fast.

```
P(C | X) = P(X | C) x P(C) / P(X)

Naive Bayes: P(X | C) = P(x1 | C) x P(x2 | C) x ... x P(xn | C); predict the class with the highest P(C) x P(X | C)
```

- **P(C)** = prior probability of the class; **P(X | C)** = likelihood; **P(C | X)** = posterior probability.
- **Laplace correction (smoothing):** add 1 to each count so that a zero count does not make the whole product zero.
- Works well in text classification and medical diagnosis; fast; but independence assumption is rarely perfectly true.

EXAMPLE: Weather data: outlook = sunny, temperature = cool, humidity = high, windy = true
• Yes class: P(yes) = 9/14; likelihoods 2/9 (sunny), 3/9 (cool), 3/9 (high), 3/9 (true). Score = 9/14 x 2/9 x 3/9 x 3/9 x 3/9 = 0.0053.
• No class: P(no) = 5/14; likelihoods 3/5, 1/5, 4/5, 3/5. Score = 5/14 x 3/5 x 1/5 x 4/5 x 3/5 = 0.0206.
• Normalise: P(yes) = 0.0053 / (0.0053 + 0.0206) = 20.5%; P(no) = 79.5%. Predict **play = no**.

Bayesian networks

A **Bayesian (belief) network** is a directed acyclic graph where each node is a random variable and each edge shows a direct dependency; each node has a **conditional probability table (CPT)** given its parents. It removes the strong independence assumption of Naive Bayes by modelling only the dependencies that exist. Example: Smoking and Family History influence Lung Cancer, which influences X-ray result. Inference computes the probability of unobserved variables given evidence. In Weka: BayesNet.

Instance-based methods: k-nearest neighbour (k-NN)

k-NN is a **lazy learner**: it stores the training instances and does no work until a new instance arrives. It finds the **k closest** training instances (by a distance function) and predicts the majority class (or the average value for numeric prediction).

```
Euclidean distance d(x, y) = square root of [ sum (xi - yi)^2 ] Manhattan distance = sum |xi - yi|
```

1. Choose k and a distance measure; normalise numeric attributes so no attribute dominates.
2. Compute the distance from the new instance to every training instance.
3. Select the k nearest neighbours.
4. Classify by majority vote (optionally weighted by 1/distance), or average for numeric targets.

- Small k is sensitive to noise; large k blurs class boundaries. Use odd k for two classes.
- Advantages: simple, no training, adapts to new data. Disadvantages: slow prediction on large data, needs normalisation, sensitive to irrelevant attributes.
- In Weka: IBk (set KNN = k).

Linear models

Linear regression predicts a numeric class as a weighted sum of attributes, with weights found by least squares (minimising the sum of squared errors).

```
y = w0 + w1 x1 + w2 x2 + ... + wn xn (simple regression: y = a + b x)

b = sum (x - xbar)(y - ybar) / sum (x - xbar)^2 a = ybar - b x xbar
```

EXAMPLE: Simple regression

• x = 1, 2, 3, 4, 5 and y = 2, 4, 5, 4, 5. xbar = 3, ybar = 4.
• Sum (x - xbar)(y - ybar) = 6 and Sum (x - xbar)^2 = 10, so b = 0.6 and a = 4 - 0.6 x 3 = 2.2.
• Model: y = 2.2 + 0.6x. For x = 6 the prediction is 5.8.

- **Logistic regression** extends the idea to classification by modelling the probability of a class with the logistic (sigmoid) function.
- Linear models need numeric attributes, assume a linear relation, and are sensitive to outliers. In Weka: LinearRegression, Logistic.

QUICK REVISION

• Naive Bayes: highest P(C) x product of P(xi | C); fix zero counts with Laplace correction.
• k-NN: lazy, distance based; normalise data and choose k carefully.
• Bayesian network = DAG + conditional probability tables. Linear regression = least squares line/plane.

Module IV: Cluster Analysis - Concepts and Methods

4.1 Basic Issues in Clustering

Definition

Clustering (cluster analysis) is the process of grouping a set of objects into clusters so that objects within a cluster are **similar** to each other and **dissimilar** to objects in other clusters. It is **unsupervised learning**: there are no predefined class labels. Business uses: customer segmentation, document grouping, image analysis, outlier detection, and market research.

Basic issues and requirements

- **Similarity or distance measure:** Euclidean, Manhattan and Minkowski distance for numeric data; matching coefficients for categorical data; cosine similarity for text.
- **Attribute types:** numeric, binary, nominal, ordinal and mixed; attributes must be normalised.
- **Scalability** to large data, ability to handle noise and outliers, discovery of arbitrary-shaped clusters.
- **Number of clusters** and input parameters: many methods need k or thresholds, which is hard to set.
- **Interpretability and usability** of the clusters for business use.
- Quality criterion: high intra-cluster similarity and low inter-cluster similarity.

Category	Idea	Examples
Partitioning	Split n objects into k groups, iteratively improve	k-means, k-medoids (PAM), EM
Hierarchical	Build a tree of clusters (dendrogram)	Agglomerative, divisive, BIRCH
Density-based	Clusters are dense regions separated by sparse regions	DBSCAN, OPTICS
Grid-based	Quantise space into cells and cluster the cells	STING, CLIQUE
Conceptual (model-based)	Describe each cluster by a concept or model	Cluster/2, Cobweb

4.2 The Cluster/2 Conceptual Clustering System

Conceptual clustering forms clusters and also gives a **symbolic description** (a concept) of each cluster, so that the clusters are explained and not only grouped by distance. **CLUSTER/2** (Michalski and Stepp) is a classic conceptual clustering system.

- It takes a set of objects described by attributes and produces a hierarchy of clusters, each described by a conjunctive concept (a combination of attribute conditions).
- Broad idea: select k seed objects; build, for each seed, descriptions that cover the seed but none of the other seeds; assign the objects to these descriptions to form clusters; evaluate the clustering with a quality criterion (simplicity of descriptions and fit); choose new seeds and repeat until the criterion stops improving.
- The same process can be applied again inside each cluster to build the hierarchy.
- Difference from distance-based methods: similarity is based on shared concepts, giving cluster descriptions people can read.

4.3 Partitioning Methods

k-means algorithm

k-means partitions n objects into k clusters; each cluster is represented by its **centroid** (the mean of its objects). It minimises the **sum of squared errors**: SSE = sum over clusters of the sum of squared distances from each object to its centroid.

1 Choose the number of clusters k and pick k initial centroids (randomly).

2. **Assign** each object to the nearest centroid (cluster with the closest mean).
3. **Update** each centroid as the mean of the objects now in its cluster.
4. Repeat assignment and update until the clusters no longer change (convergence) or a maximum number of iterations is reached.

EXAMPLE: k-means with k = 2 on values 2, 3, 4, 10, 11, 12, 20, 25, 30
• Start with centroids m1 = 2 and m2 = 4. Iteration 1: C1 = {2, 3}, C2 = {4, 10, 11, 12, 20, 25, 30}; means 2.5 and 16.
• Iteration 2: C1 = {2, 3, 4}, C2 = {10, 11, 12, 20, 25, 30}; means 3 and 18. Iteration 3: 10 moves to C1; C1 = {2, 3, 4, 10}, C2 = {11, 12, 20, 25, 30}; means 4.75 and 19.6.
• Iteration 4: 11 and 12 move to C1; C1 = {2, 3, 4, 10, 11, 12} (mean 7), C2 = {20, 25, 30} (mean 25). Iteration 5: no change, so stop.
• Final clusters: {2, 3, 4, 10, 11, 12} and {20, 25, 30}.

- Strengths: simple, fast; complexity about O(n k t) for t iterations.
- Weaknesses: must specify k; sensitive to initial centroids (may reach a local optimum; run several times); sensitive to outliers because the mean is pulled by extreme values; finds roughly spherical clusters; only for numeric data.
- Weka: SimpleKMeans (set numClusters and seed).

k-medoids (PAM: Partitioning Around Medoids)

k-medoids is like k-means but each cluster is represented by a **medoid**, an actual, most centrally located object of the cluster, instead of a mean. This makes it more robust to noise and outliers.

1. Choose k objects at random as the initial medoids.
2. Assign every other object to the nearest medoid.
3. For each medoid m and each non-medoid o, try swapping them and compute the change in total cost (sum of distances of objects to their medoid).
4. If a swap reduces the total cost, make the swap.
5. Repeat until no swap improves the cost.

Aspect	k-means	k-medoids (PAM)
Cluster centre	Mean (centroid); may not be a real data point	Medoid; always an actual data object
Sensitivity to outliers	High (mean is pulled)	Low (more robust)
Cost / cluster quality	Sum of squared Euclidean distances	Sum of absolute dissimilarities to the medoid
Data types	Numeric attributes (needs a mean)	Any data with a distance or dissimilarity matrix
Efficiency	Fast and scalable: O(nkt)	Slower, about O(k(n-k)^2) per iteration; CLARA and CLARANS scale it
Number of clusters	k must be given	k must be given

QUICK REVISION
• k-means: assign to nearest centroid, recompute means, repeat. k-medoids: centres are real objects, so more outlier-robust but slower.
• Both need k in advance and find convex clusters.

Expectation-Maximisation (EM)

EM is a model-based (probabilistic) extension of k-means. Instead of a hard assignment, each object has a **probability of belonging to each cluster** (soft clustering). It assumes data comes from a mixture of distributions (typically Gaussian), each with its own mean and variance.

1. **Initialise** the parameters (means, variances, cluster weights) for k clusters.

2. **E-step (Expectation):** compute for each object the probability (responsibility) of belonging to each cluster given current parameters.
3. **M-step (Maximisation):** re-estimate the parameters to maximise the likelihood using those probabilities as weights.
4. Repeat E and M steps until the log-likelihood converges.

- Advantages: soft membership, flexible cluster shapes (via variances), can estimate k by cross-validation (Weka does this if numClusters = -1).
- Disadvantages: slower than k-means; can converge to local optimum; assumes a distribution form.
- In Weka: EM.

4.4 Hierarchical Methods

Hierarchical clustering builds a tree of nested clusters called a **dendrogram**; cutting the tree at a chosen height gives a clustering. The number of clusters need not be fixed in advance.

- **Agglomerative (bottom-up, AGNES):** start with each object as its own cluster; repeatedly merge the two closest clusters until one cluster remains (or a stopping condition).
- **Divisive (top-down, DIANA):** start with all objects in one cluster; repeatedly split the most heterogeneous cluster until each object is separate (or a condition is met).

Linkage (distance between clusters)	Definition	Behaviour
Single linkage	Minimum distance between members of the two clusters	Can produce long chains
Complete linkage	Maximum distance between members	Compact clusters, sensitive to outliers
Average linkage	Average of all pairwise distances	Compromise
Centroid / Ward	Distance between centroids / minimum increase in variance	Tends to give round clusters

- Advantages: no need to choose k first; dendrogram is informative. Disadvantages: merges and splits cannot be undone (greedy); at least O(n^2) cost, so not for very large data.
- **BIRCH** and **CURE** improve scalability. Weka has HierarchicalClusterer.

4.5 Density-Based Methods

Clusters are **dense regions** of objects separated by regions of low density. They find **arbitrary-shaped clusters** and handle noise well.

DBSCAN

- **Epsilon (eps):** radius of the neighbourhood. **MinPts:** minimum number of points required in the eps-neighbourhood to be dense.
- **Core point:** has at least MinPts points within eps. **Border point:** within eps of a core point but not core itself. **Noise (outlier):** neither.
- **Density-reachable:** reached through a chain of core points.

1. Pick an unvisited point and find its eps-neighbourhood.
2. If it has at least MinPts points it is a core point: start a new cluster and expand it by adding all density-reachable points.
3. Otherwise mark it as noise (it may later become a border point).
4. Repeat until all points are visited.

Advantages: no need to specify k, finds arbitrary shapes, identifies outliers. Disadvantages: sensitive to eps and MinPts, struggles with clusters of very different densities. **OPTICS** orders points to reveal clusters at multiple densities. Weka has DBSCAN (as an optional package).

4.6 Grid-Based Methods

Grid-based methods **quantise the data space into a finite number of cells** forming a grid, and all clustering is done on the grid cells. Processing time depends on the number of cells and not on the number of objects, so they are very fast.

- **STING** (Statistical Information Grid): hierarchical grid of rectangular cells storing statistics (count, mean, min, max); answers queries top-down.
- **CLIQUE:** grid and density-based subspace clustering for high-dimensional data.
- **WaveCluster:** uses wavelet transform on the grid.
- Advantages: speed, easy parallelisation. Disadvantage: quality depends on cell size; cluster boundaries are horizontal or vertical only.

4.7 Conceptual Clustering: Cobweb

Cobweb is an **incremental**, hierarchical conceptual clustering algorithm. It builds a **classification tree** in which each node is a concept with probabilistic attribute-value descriptions. Instances are added one at a time and the tree adapts.

- For each new instance, Cobweb considers four operators: place it in an **existing node**, create a **new node**, **merge** two nodes, or **split** a node.
- It chooses the operator giving the highest **category utility (CU)**, a measure that rewards clusters where attribute values are predictable within a cluster and distinguishable between clusters.
- Merge and split let Cobweb recover from a bad ordering of instances.
- Control parameters in Weka: **acuity** (minimum standard deviation for numeric attributes) and **cutoff** (minimum category utility to avoid unlimited growth of the tree).
- Limitations: results depend on instance order; the assumption of independent attributes; trees can become large.

EXAMPLE: Weka clustering experiments (Module IV lab)
• Cluster tab: choose SimpleKMeans (k = 2 or 3), EM, or Cobweb; choose "Classes to clusters evaluation" if the dataset has a class.
• Read the output: cluster centroids, instances per cluster, and for evaluation the incorrectly clustered instances percentage.
• Right-click the result list and choose Visualize cluster assignments to see the groups; interpret each cluster in plain business language (for example "young high-income buyers").

Method	Needs k?	Cluster shape	Handles outliers	Best for
k-means	Yes	Spherical	Poor	Large numeric data, quick segmentation
k-medoids	Yes	Spherical	Good	Smaller data with outliers
EM	Yes (or estimated)	Elliptical / probabilistic	Moderate	Soft membership
Hierarchical	No (cut dendrogram)	Flexible by linkage	Moderate	Small data, taxonomy
DBSCAN	No	Arbitrary	Good (marks noise)	Spatial data with noise
Grid-based	No	Arbitrary (grid-aligned)	Moderate	Very large data, speed

Method	Needs k?	Cluster shape	Handles outliers	Best for
Cobweb	No	Hierarchy of concepts	Moderate	Incremental, explainable clusters

Module V: Advanced Techniques, Data Mining Software and Applications

5.1 Text Mining

Definition

Text mining (text data mining) extracts useful patterns and knowledge from **unstructured text**, such as emails, reviews, news, social media and documents. Text must first be converted into a structured form so that standard mining algorithms can be used. Applications: spam filtering, sentiment analysis, document classification, customer feedback analysis, and information retrieval.

Text preprocessing

- **Tokenisation:** split text into words (tokens).
- **Stop-word removal:** drop very common words (the, is, of).
- **Stemming / lemmatisation:** reduce words to a root (running, runs to run).
- **Case folding and cleaning:** lowercase, remove punctuation and numbers.

Extracting attributes (keywords)

Each document is turned into a vector of attributes where each attribute is a word or keyword (the **bag-of-words** model: word order is ignored). Not all words are equally useful, so keywords are selected by frequency or weighting.

```
TF(t, d) = count of term t in document d / total terms in d

IDF(t) = log ( N / number of documents containing t ) TF-IDF = TF x IDF
```

A high TF-IDF means the word is frequent in this document but rare across the collection, so it is a good keyword. Attribute selection (information gain, chi-square) reduces the huge number of word attributes. In Weka: the StringToWordVector filter converts text to word attributes.

Structural approaches: parsing and soft parsing

- **Parsing:** analyses the grammatical structure of sentences (part-of-speech tagging, syntax trees) to identify relations such as subject-verb-object. It is accurate but costly and fails on ungrammatical text.
- **Soft parsing (shallow or partial parsing):** a tolerant, partial analysis that identifies only useful chunks (noun phrases, named entities, key patterns) without a full grammar. It is faster, robust to errors and informal text, and good enough for information extraction.
- Use: extracting entities such as names, dates, products and relationships from text to build structured attributes.

The Bayesian approach to classifying text

Text classification assigns a document to a category (for example spam or not spam, positive or negative). The **Naive Bayes** classifier is popular because it is fast and works well with many attributes.

```
P(c | d) is proportional to P(c) x product over words w in d of P(w | c)

P(w | c) = (count of w in class c + 1) / (total words in class c + vocabulary size)
(Laplace smoothing)
```

1. Estimate the prior P(c) as the fraction of training documents in class c.
2. Estimate P(w | c) for every word, with Laplace smoothing to avoid zero probabilities.
3. For a new document multiply P(c) by P(w | c) for each word (use logarithms to avoid underflow) and choose the class with the highest score.

EXAMPLE: Spam filter
• Training: 3 spam emails and 1 normal email. Suppose "offer" appears often in spam and "meeting" appears in normal mail.
• A new email containing "offer" scores higher for spam because P(offer | spam) is large; the classifier predicts spam.
• Weka: StringToWordVector + NaiveBayesMultinomial.

QUICK REVISION
• Text mining: preprocess (tokenise, stop words, stem), build keyword attributes (bag of words, TF-IDF), then classify or cluster.
• Parsing = full grammatical analysis; soft parsing = partial, robust analysis of useful chunks.
• Bayesian text classification = Naive Bayes over word probabilities with Laplace smoothing.

5.2 Web Mining

Web mining applies data mining to discover patterns from web data: page content, link structure and user behaviour.

Category	Data mined	Examples
Web content mining	Text, images, multimedia on pages	Page classification, product extraction, opinion mining
Web structure mining	Hyperlinks between pages	PageRank, HITS (hubs and authorities), finding communities
Web usage mining	Server logs, clickstream, sessions	Recommendation, personalisation, site improvement

Classifying web pages

- Assign pages to categories (for example news, shopping, education) using page content features (keywords, title, headings, meta tags), HTML structure and link features (neighbouring pages' classes).
- Methods: Naive Bayes, decision trees, k-NN, support vector machines on word vectors; links help because linked pages tend to share topics.
- Uses: search engines, directories, content filtering, targeted advertising.

Extracting knowledge from the web

- **Crawling:** spiders follow links to collect pages.
- **Information extraction and wrappers:** programs that pull structured data (prices, names, dates) from semi-structured HTML pages; wrapper induction learns them from examples.
- **Mining usage logs:** clean logs, identify users and sessions, find access patterns with association or sequential pattern mining, and apply them (for example product recommendations).
- **Challenges:** huge and dynamic data, noisy and duplicate pages, heterogeneous formats, and privacy and ethical issues (user tracking, consent, data protection).

5.3 Data Mining Software and Applications

Data mining software

Tool	Type	Notes
Weka	Open source (Java), GUI	Used in this course; preprocessing, classification, clustering, association, visualisation
RapidMiner	GUI workflow, commercial and free edition	Drag-and-drop process design
KNIME	Open source workflow	Visual pipelines; integrates Python and R
Orange	Open source visual programming	Good for teaching and visualisation
R and Python (scikit-learn, pandas)	Programming languages and libraries	Flexible, widely used in industry
SAS Enterprise Miner, IBM SPSS Modeler	Commercial suites	Enterprise scale and support
Power BI, Tableau	BI and visualisation	Dashboards and reporting of mined results

Selecting software for a business problem: consider the data volume and type, required techniques, ease of use and skills available, integration with existing systems, scalability and performance, visualisation and reporting, cost and licensing, and vendor or community support.

Applications of data mining

Domain	Application
Retail and e-commerce	Market basket analysis, recommendation, customer segmentation, demand forecasting
Banking and finance	Credit scoring, fraud detection, loan default prediction, risk management
Telecom	Churn prediction, usage clustering, network fault detection
Marketing and CRM	Targeted campaigns, customer lifetime value, response modelling
Healthcare	Diagnosis support, patient grouping, treatment effectiveness
Insurance	Claim fraud, premium pricing
Manufacturing and supply chain	Quality control, predictive maintenance, inventory optimisation
Human resources	Attrition prediction, talent segmentation
Web and social media	Sentiment analysis, search ranking, personalisation

Ethical and practical issues

- Privacy and data protection, informed consent, bias and fairness in models, transparency of decisions, and data security.
- Course expectations: acknowledge dataset sources and any external code; follow ethical data practices and maintain data integrity.

Quick Revision Section

R1. Formula Sheet

Topic	Formula
Min-max normalisation	v' = (v - min) / (max - min)
Z-score	v' = (v - mean) / std dev
Decimal scaling	v' = v / 10^j
Support	count(A and B) / N
Confidence	support(A and B) / support(A)
Lift	confidence(A => B) / support(B)
Entropy	Info(D) = - sum pi log2 pi
Information gain	Gain(A) = Info(D) - Info_A(D)
Gini	1 - sum pi^2
Accuracy	(TP + TN) / total
Precision, recall	TP/(TP+FP), TP/(TP+FN)
Bayes theorem	P(C|X) = P(X|C) P(C) / P(X)
Euclidean distance	square root of sum (xi - yi)^2
Simple regression	b = sum (x - xbar)(y - ybar) / sum (x - xbar)^2; a = ybar - b xbar
TF-IDF	TF x log(N / df)
IQR	Q3 - Q1

R2. Key Comparisons

Pair	Difference
Data mining vs OLAP	Mining is discovery-driven and automatic; OLAP is user-driven, interactive and verifies hypotheses.
OLTP vs data warehouse	OLTP supports daily transactions; warehouse supports analysis on historical integrated data.
Classification vs clustering	Classification is supervised (known labels); clustering is unsupervised (no labels).
Classification vs prediction	Classification predicts a class label; (numeric) prediction estimates a continuous value.
Roll-up vs drill-down	Roll-up summarises to higher level; drill-down goes to more detail.
Slice vs dice	Slice fixes one dimension; dice fixes two or more.
Decision tree vs covering rules	Divide-and-conquer on all classes vs separate-and-conquer one class at a time.
k-means vs k-medoids	Mean centre vs actual-object centre; medoids are more robust to outliers but slower.
Agglomerative vs divisive	Bottom-up merging vs top-down splitting.
Parsing vs soft parsing	Full grammatical analysis vs partial, robust analysis.
Star vs snowflake schema	Denormalised dimensions vs normalised dimensions.
Equal-width vs equal-frequency binning	Same interval size vs same number of values per bin.
Support vs confidence	Frequency of the whole rule vs reliability of the consequent given the antecedent.

R3. Algorithm Cheat Sheet (steps in one line)

Algorithm	Steps
Apriori	Find L1, join to make Ck, prune with Apriori property, count, repeat; then generate rules by confidence.
1R	For every attribute make a rule per value (majority class), count errors, choose the attribute with the least error.
ID3 / J48	Choose attribute with highest information gain (gain ratio), split, recurse, prune.
PRISM	For each class add the test with highest accuracy until perfect, remove covered instances, repeat.
Naive Bayes	Compute priors and likelihoods, multiply, normalise, choose highest posterior.
k-NN	Compute distances, take k nearest, vote.
k-means	Pick k centres, assign to nearest, recompute means, repeat until stable.
k-medoids (PAM)	Pick k medoids, assign, try swaps that reduce cost, repeat.
EM	Initialise parameters, E-step (probabilities), M-step (re-estimate), repeat until convergence.
Agglomerative	Each object a cluster, merge closest pair repeatedly, build dendrogram.
DBSCAN	Find core points (eps, MinPts), grow clusters through density-reachable points, mark noise.
Cobweb	Insert instances one by one using new/merge/split/place by category utility.
AOI	Collect data, remove or generalise attributes, apply thresholds, merge tuples with counts, present.

R4. Likely Exam Questions

- Define data mining and explain the stages of the data mining (KDD) process.
- Differentiate data mining from OLAP, DBMS, machine learning and statistics.
- Explain OLAP operations with an example from the loan dataset.
- What is data cleaning? Why is it important? Explain methods for handling missing and noisy data.
- Explain data transformation and normalisation with numerical examples.
- Explain discretisation and concept hierarchies.
- Explain interestingness measures. Explain attribute-oriented induction with an example.
- Explain the Apriori algorithm with an example; calculate support, confidence and lift.
- Explain the 1R algorithm, decision tree induction (information gain), and covering rules.
- Explain Naive Bayes with a numerical example; explain Bayesian networks and k-nearest neighbour.
- Explain k-means with a numerical example; compare k-means and k-medoids.
- Explain hierarchical, density-based and grid-based clustering; explain Cobweb.
- Explain text mining and the Bayesian approach to text classification.
- Explain web mining and its categories; list data mining software and applications.

R5. Glossary

Term	Meaning
KDD	Knowledge Discovery from Data: whole process from raw data to knowledge
Attribute	A column or feature describing an instance
Instance	A row or record
Class attribute	The attribute to be predicted
Overfitting	Model too closely fits the training data and performs poorly on new data
Cross-validation	Splitting data into k folds and rotating the test fold to estimate accuracy
Entropy	A measure of impurity or disorder in a set
Centroid	Mean point of a cluster
Medoid	Most central actual object of a cluster
Dendrogram	Tree diagram of hierarchical clustering
Outlier	A value or object that deviates greatly from the rest
Concept hierarchy	Mapping from low-level to higher-level concepts
Bag of words	Text representation as unordered word counts
Lazy learner	Algorithm that postpones learning until a query arrives (k-NN)
ARFF	Weka's Attribute-Relation File Format